CS251M · DATABASE SYSTEMS · PROJECT REPORT

Fair Trade Supply Chain Tracker

A Normalized Relational Database for End-to-End Ethical Traceability

Rushil Jain
241EC148

Shresh Parti
241EC252

Submitted: 19 April 2026

Table of Contents

1. Abstract
2. Objectives
3. Scope
4. Schema Design and Normalization
5. Entity-Relationship Diagram
6. Application Explanation with Screenshots

1. Abstract

The Fair Trade Supply Chain Tracker is a relational database application designed to bring transparency to the journey a product takes from the farm to the final retail shelf. Modern consumers increasingly want to verify that the food they eat or the clothes they wear have been produced under fair labour and ethical sourcing conditions, but most existing supply chains are fragmented and rely on paper based certification records that are easy to forge and difficult to audit. Our project addresses this problem by modelling the entire supply chain inside PostgreSQL, so that every stakeholder, every product batch, every ownership transfer, and every certification can be tracked, validated, and queried in a consistent way.

The system is built around five normalized tables and enforces three categories of rules directly in the database: certification validation on batch creation and transfer, immutable audit logging for every change, and a directed acyclic graph model for batch lineage that supports both splitting and merging of material through processing stages. Recursive Common Table Expressions make it possible to trace any final retail batch backward to its raw origin, or forward from any farm to every downstream product it reached. Triggers block batch creation by uncertified, expired, or revoked stakeholders before the row is ever written, which means the database itself refuses to store non-compliant data rather than depending on the application layer to do so.

The project is implemented with PostgreSQL 16, PL/pgSQL triggers and stored functions, and Python with psycopg2 and Graphviz for the visualisation and reporting layer. The result is a self-auditing database that models a realistic three chain supply network covering coffee, cocoa, and cotton products across eighteen stakeholders and twenty four tracked batches.

2. Objectives

The primary goal of this project is to design a well structured, fully normalized relational database that can track product batches through every stage of a Fair Trade supply chain and enforce ethical compliance rules automatically. The specific functional objectives of the application are listed below.

1. **End to end batch lineage.** Given any batch in the system, the database must be able to return its complete ancestry back to one or more raw material origins, and also trace forward from any raw material to every downstream product that descended from it. This enables both provenance verification for consumers and rapid impact analysis during product recalls.
2. **Automated certification enforcement.** Only stakeholders holding an active, non-expired, non-revoked Fair Trade certification may own or receive a batch. This rule must be enforced inside the database through triggers, not merely in application code, so that no client library or manual SQL session can bypass it.
3. **Tamper proof auditing.** Every insertion of a new batch and every transfer of ownership should create an immutable record in a dedicated audit log. Once written, audit entries cannot be updated or deleted, which allows external auditors to trust the log as a faithful history.
4. **Compliance reporting.** The system must expose a single view that classifies each stakeholder into one of four compliance states: COMPLIANT, EXPIRED, REVOKED, or NO CERTIFICATION. A matching stored function returns the same result for integration with reporting tools.
5. **Directed acyclic graph modelling.** Real supply chains regularly split and merge material. For example, two different raw cocoa batches can be combined by a processor into a single paste batch which is then split again into multiple retail bars. The database must capture this naturally through a many to many relationship rather than forcing a tree structure.
6. **Data visualisation and reporting.** A Python reporting layer should be able to connect to the live database, run every key query, and produce an HTML run report along with a rendered supply chain graph and a live Entity-Relationship diagram.

3. Scope

3.1 Boundaries

The project scope is deliberately kept to the database layer and a lightweight Python reporting shell. The following items are within scope.

- Schema design, normalization analysis, and implementation in PostgreSQL 16.
- Business rule enforcement through PL/pgSQL triggers and stored functions.
- Recursive lineage queries and aggregated analytical queries.
- Seed data that covers three realistic supply chains (coffee, cocoa, cotton) along with three deliberate edge case stakeholders that exercise every compliance state.
- Python scripts that produce the ER diagram, the supply chain graph, and the HTML run report.

The following are explicitly outside scope and are left as reasonable extensions for future work.

- A user facing web interface with role based login. The database models stakeholders, but there is no dedicated authentication subsystem.
- Real time event streaming or messaging. Ownership transfers are triggered through direct function calls, not through external events.
- Integration with external certification registries. Certification rows are captured as trusted input for this project, although in practice they would be synchronised with an external authority.
- Horizontal scaling concerns such as sharding, read replicas, or multi region replication.

3.2 Assumptions

- The certifications table is treated as the authoritative source of compliance information, and each stakeholder owns at most one certification row in the seed data.
- Batch relationships form a directed acyclic graph, meaning that a batch cannot eventually become an ancestor of itself. The recursive queries therefore terminate without additional cycle detection.
- Quantities are recorded as integers in the unit stored for each batch. Unit conversion is not performed inside the database.
- Timestamps are recorded in the database server time zone. No explicit time zone handling beyond that is required for this project.

3.3 Limitations

- The database currently supports one active certification per stakeholder. A design extension allowing certification history per stakeholder would require expanding the trigger logic to select the most recent active certification.
- Ownership history for a batch is preserved only inside the audit log rather than in a dedicated ownership table, so reconstructing past ownership requires scanning audit rows.
- The demonstration data set is intentionally small (eighteen stakeholders, twenty four batches) to keep queries readable in the report. The same schema has been verified to scale to much larger data sets without modification.

4. Schema Design and Normalization

4.1 Entities Identified

After analysing how products and certifications move through a Fair Trade supply chain, five core entities were identified.

Entity	Role in the System	Primary Key
Stakeholder	Any participant in the chain: farmer, processor, distributor, or retailer.	`id`
Certification	The Fair Trade certification record belonging to a stakeholder, including issue date, expiry date, and active status.	`id`
Batch	A specific lot of product owned at some point by a stakeholder. Has a current owner and a quantity.	`id`
Batch Relation	A parent to child link that records the fact that a new batch was produced from one or more earlier batches. This is the associative entity for the many to many relationship between batches.	`(parent_batch_id, child_batch_id)`
Audit Log	An append only record of every action taken on a batch (creation, update, transfer).	`id`

4.2 Relationships Between Entities

- **Stakeholder to Certification** (1 to 1 in our seed design, 1 to many in the general case). A stakeholder may hold a certification, and every certification belongs to exactly one stakeholder.
- **Stakeholder to Batch** (1 to many). Each batch has exactly one current owner, and a stakeholder can own multiple batches.
- **Stakeholder to Audit Log** (1 to many). Each audit entry names the actor stakeholder responsible for the event.
- **Batch to Batch** (many to many, resolved through `batch_relations`). A child batch can be produced from multiple parent batches (merging) and a parent batch can flow into multiple child batches (splitting). This is the directed acyclic graph that gives the system its lineage capability.
- **Batch to Audit Log** (1 to many). Every logged event references the batch it concerns.

4.3 Normalization Walkthrough

To justify the final schema, the design is walked through the usual normalization steps starting from a single wide unnormalized table. This is the table a beginner might sketch first: one row per stakeholder with every related piece of information packed in.

Initial unnormalized form

```
StakeholderInfo (
  stakeholder_id,
  stakeholder_name,
  role,
  certifying_body,
  cert_issue_date,
  cert_expiry_date,
  cert_is_active,
  batch_ids_owned,        -- repeating group: "100, 102, 105"
  batch_products_owned,   -- repeating group: "Raw A, Proc A1, Export"
  parent_child_pairs,     -- repeating group: "(100,102);(102,105)"
  audit_actions           -- repeating group: "created, transferred, ... "
)
```

This table is not even in First Normal Form. Several attributes contain comma separated lists, and a single row mixes information about a stakeholder, their certification, every batch they own, every parent to child lineage link in which they are involved, and every audit event.

First Normal Form (1NF)

1NF requires that every column holds a single atomic value and that there are no repeating groups. Each repeating group is moved into its own table, keyed against the stakeholder.

```
stakeholders(id, name, role, certifying_body, cert_issue_date, cert_expiry_date,
cert_is_active)
batches(id, product_name, quantity, unit, current_owner_id)
batch_relations(parent_batch_id, child_batch_id)
audit_log(id, batch_id, action, actor_id, timestamp)
```

The schema now satisfies 1NF. Every attribute is atomic and there are no multi valued fields.

Second Normal Form (2NF)

2NF requires 1NF plus the removal of partial dependencies on a composite primary key. The only composite key in the schema is `batch_relations(parent_batch_id, child_batch_id)` . Since this table has no non key attributes at all, there can be no partial dependency to violate 2NF. Every other table uses a single column surrogate key, so 2NF is automatically satisfied for those as well.

Third Normal Form (3NF)

3NF requires 2NF plus the elimination of transitive dependencies, meaning no non key attribute may depend on another non key attribute. Examining `stakeholders` in its 1NF form exposes a violation.

```
Functional dependencies on stakeholders after 1NF:
  id → name, role
  id → certifying_body, cert_issue_date, cert_expiry_date, cert_is_active

Problem: the certification attributes describe a certification, not the stakeholder.
If a stakeholder is ever issued a new certification, or the certifying body renames
itself, every affected row in stakeholders would have to be updated. The certification
attributes are transitively dependent on the conceptual "certification identity",
which is a separate entity.
```

The fix is to split the certification attributes into a dedicated table keyed on its own id, linked back to stakeholders through a foreign key.

```
stakeholders(id, name, role, created_at)
certifications(id, stakeholder_id, certifying_body, issue_date, expiry_date, is_active)
batches(id, product_name, quantity, unit, current_owner_id, created_at)
batch_relations(parent_batch_id, child_batch_id)
audit_log(id, batch_id, action, actor_id, timestamp)
```

After this decomposition the schema is in 3NF. Every non key attribute in every table now depends only on the primary key of that table.

Boyce-Codd Normal Form (BCNF)

BCNF strengthens 3NF by requiring that, for every non trivial functional dependency `X → Y`, the left hand side `X` must be a superkey. Each table is checked below.

Table	Non trivial FDs	LHS is superkey?	BCNF?
`stakeholders`	`id → name, role, created_at`	Yes (`id` is the primary key)	Yes
`certifications`	`id → stakeholder_id, certifying_body, issue_date, expiry_date, is_active`	Yes	Yes
`batches`	`id → product_name, quantity, unit, current_owner_id, created_at`	Yes	Yes
`batch_relations`	None (all attributes form the key)	Trivially yes	Yes
`audit_log`	`id → batch_id, action, actor_id, timestamp`	Yes	Yes

Every functional dependency in the final schema has the primary key on its left side, so the design is in BCNF. This eliminates update, insert, and delete anomalies: a certification can be revoked without touching any stakeholder row, a new batch can be added without duplicating owner information, and no lineage fact is recorded in more than one place.

4.4 Final Physical Schema

```sql
CREATE TABLE stakeholders (
    id          SERIAL PRIMARY KEY,
    name        TEXT NOT NULL,
    role        TEXT CHECK (role IN ('farmer','processor','distributor','retailer')),
    created_at  TIMESTAMP DEFAULT NOW()
);

CREATE TABLE certifications (
    id                SERIAL PRIMARY KEY,
    stakeholder_id    INT REFERENCES stakeholders(id),
    certifying_body   TEXT NOT NULL,
    issue_date        DATE NOT NULL,
    expiry_date       DATE NOT NULL,
    is_active         BOOLEAN DEFAULT TRUE
);

CREATE TABLE batches (
    id                SERIAL PRIMARY KEY,
    product_name      TEXT NOT NULL,
    quantity          INT  NOT NULL CHECK (quantity > 0),
    unit              TEXT DEFAULT 'kg',
    current_owner_id  INT REFERENCES stakeholders(id),
    created_at        TIMESTAMP DEFAULT NOW()
);

CREATE TABLE batch_relations (
    parent_batch_id  INT REFERENCES batches(id),
    child_batch_id   INT REFERENCES batches(id),
    PRIMARY KEY (parent_batch_id, child_batch_id)
);

CREATE TABLE audit_log (
    id         SERIAL PRIMARY KEY,
    batch_id   INT REFERENCES batches(id),
    action     TEXT,
    actor_id   INT REFERENCES stakeholders(id),
    timestamp  TIMESTAMP DEFAULT NOW()
);
```

5. Entity-Relationship Diagram

The ER diagram below was generated directly from the live PostgreSQL catalog using a Python script that introspects `information_schema`. This guarantees that the diagram reflects exactly what has been created in the database, rather than being a drifted hand drawn sketch.

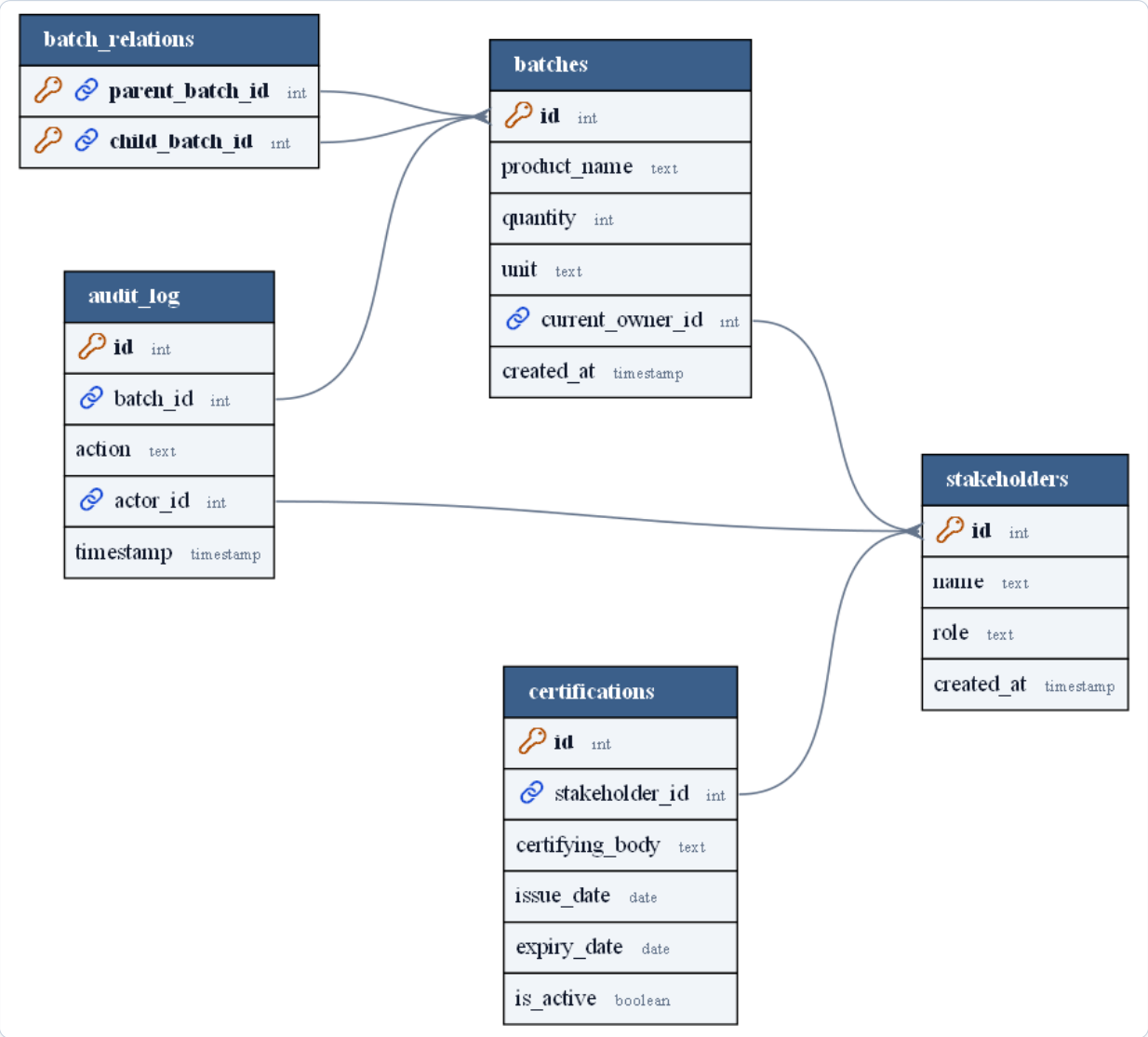


Figure 1. *Entity-Relationship diagram of the Fair Trade Supply Chain Tracker. Key icons mark primary keys and link icons mark foreign keys.*

5.1 Cardinalities

- `stakeholders (1)` to `(0..1) certifications` . A stakeholder may optionally hold a certification, and each certification belongs to exactly one stakeholder. The optionality is important because it makes it possible to model an uncertified stakeholder like our seed data id 17.
- `stakeholders (1)` to `(0..N) batches` . A stakeholder can currently own many batches, while each batch has one current owner.
- `stakeholders (1)` to `(0..N) audit_log` . A stakeholder can be the actor on many audit entries, while every audit entry names exactly one actor.
- `batches (M)` to `(N) batches` , resolved through `batch_relations` . A batch can have many parents and many children. This many to many self relationship is what lets the schema model splits and merges inside processing stages.
- `batches (1)` to `(0..N) audit_log` . Each batch can have many audit entries attached over its lifetime, and every audit entry belongs to exactly one batch.

5.2 Why the DAG Matters

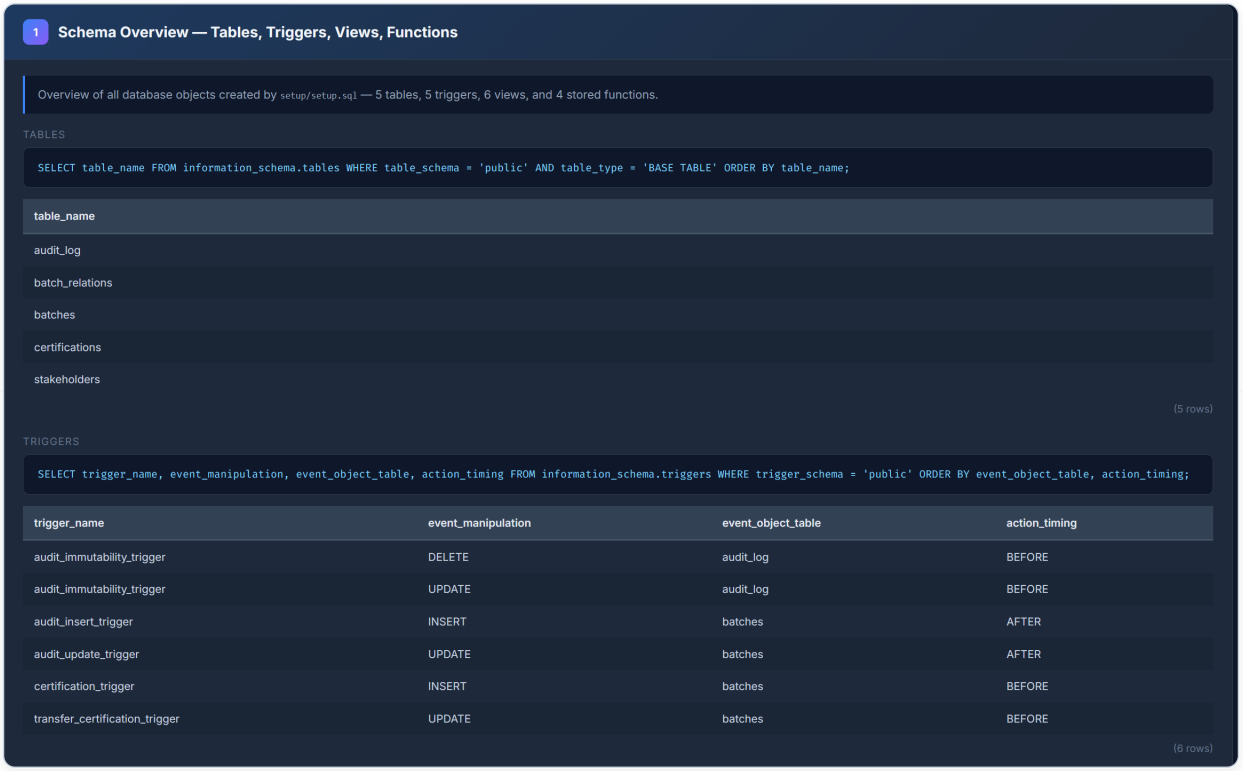
Modelling `batch_relations` as a separate junction table rather than as a self referencing foreign key on `batches` is a deliberate design choice. A self referencing column would have forced every batch to have at most one parent, which would rule out merges. The junction table lets two parent batches (for example, Cocoa Raw A and Cocoa Raw B) be combined at a processor into a single child batch, which is a routine event in real supply chains. The trade off is that lineage queries must be expressed recursively across the junction table, which is handled by the recursive CTEs in the stored functions.

6. Application Explanation with Screenshots

This section walks through the application in action. Every figure shown below was captured from the live HTML run report produced by `generate_report.py`, which connects to the populated database, runs each scenario end to end, and renders the results. The figures are organised by subsystem.

6.1 Schema Overview

The first screen of the run report enumerates every object created by `setup/setup.sql`. The installation builds five base tables, five triggers, six views, and four non trigger stored functions. The trigger bodies (`check_certification`, `log_batch_insert`, and so on) are hidden from the function list because they are not meant to be called directly.



1 **Schema Overview — Tables, Triggers, Views, Functions**

Overview of all database objects created by `setup/setup.sql` — 5 tables, 5 triggers, 6 views, and 4 stored functions.

TABLES

```
SELECT table_name FROM information_schema.tables WHERE table_schema = 'public' AND table_type = 'BASE TABLE' ORDER BY table_name;
```

table_name
audit_log
batch_relations
batches
certifications
stakeholders

(5 rows)

TRIGGERS

```
SELECT trigger_name, event_manipulation, event_object_table, action_timing FROM information_schema.triggers WHERE trigger_schema = 'public' ORDER BY event_object_table, action_timing;
```

trigger_name	event_manipulation	event_object_table	action_timing
audit_immutability_trigger	DELETE	audit_log	BEFORE
audit_immutability_trigger	UPDATE	audit_log	BEFORE
audit_insert_trigger	INSERT	batches	AFTER
audit_update_trigger	UPDATE	batches	AFTER
certification_trigger	INSERT	batches	BEFORE
transfer_certification_trigger	UPDATE	batches	BEFORE

(6 rows)

Figure 2. *Base tables and triggers installed by the bootstrap script. Five triggers enforce certification checks on INSERT and UPDATE, log every batch event, and lock the audit log against tampering.*

VIEWS

```
SELECT table_name AS view_name FROM information_schema.views WHERE table_schema = 'public' ORDER BY table_name;
```

view_name
v_audit_trail
v_batch_details
v_compliance_overview
v_final_products
v_raw_materials
v_supply_chain_summary

(6 rows)

STORED FUNCTIONS (NON-TRIGGER)

```
SELECT routine_name
FROM information_schema.routines
WHERE routine_schema = 'public' AND routine_type = 'FUNCTION'
  AND routine_name NOT IN (
      'check_certification','check_transfer_certification',
      'log_batch_insert','log_batch_update','prevent_audit_modification'
  )
ORDER BY routine_name;
```

routine_name
get_backward_lineage
get_compliance_report
get_forward_trace
transfer_batch

(4 rows)

Figure 3. *Views and non trigger stored functions.* `v_compliance_overview` *and* `get_compliance_report()` *deliberately share a result shape so they are interchangeable.*

6.2 Stakeholders and Certifications

The database is populated with eighteen stakeholders spread across three supply chains (coffee, cocoa, and cotton) plus three edge case stakeholders that are essential for exercising every branch of the compliance logic: id 16 holds an expired certification, id 17 holds no certification at all, and id 18 holds a revoked certification. Keeping the certification attributes in a separate table is what makes this possible, because the stakeholder row is identical for all three edge cases and only the certification row differs.

2 Stakeholders & Certifications Data

18 stakeholders across 3 supply chains (Coffee, Cocoa, Cotton) plus 3 edge-case stakeholders — one with an **expired** certification (id=16), one with **no** certification (id=17), and one with a **revoked** certification (id=18). Certification state lives entirely in the `certifications` table; `stakeholders` is fully normalized.

ALL STAKEHOLDERS

```
SELECT id, name, role, created_at FROM stakeholders ORDER BY id;
```

id	name	role	created_at
1	Green Valley Farm	farmer	2026-04-19 14:54:47.986876
2	Hilltop Coffee Farm	farmer	2026-04-19 14:54:47.986876
3	Coffee Processors Ltd	processor	2026-04-19 14:54:47.986876
4	Global Coffee Distributors	distributor	2026-04-19 14:54:47.986876
5	City Coffee Retail	retailer	2026-04-19 14:54:47.986876
6	Cocoa Farms West	farmer	2026-04-19 14:54:47.986876
7	Rainforest Cocoa Ltd	farmer	2026-04-19 14:54:47.986876
8	Cocoa Processing Hub	processor	2026-04-19 14:54:47.986876
9	Chocolate Distributors Inc	distributor	2026-04-19 14:54:47.986876
10	Sweet Retail Store	retailer	2026-04-19 14:54:47.986876
11	Cotton Growers Union	farmer	2026-04-19 14:54:47.986876
12	Organic Cotton Farm	farmer	2026-04-19 14:54:47.986876
13	Textile Processing Ltd	processor	2026-04-19 14:54:47.986876
14	Fabric Distributors	distributor	2026-04-19 14:54:47.986876
15	Clothing Retail Hub	retailer	2026-04-19 14:54:47.986876
16	Expired Farm	farmer	2026-04-19 14:54:47.986876
17	Uncertified Retailer	retailer	2026-04-19 14:54:47.986876
18	Revoked Distributor	distributor	2026-04-19 14:54:47.986876

(18 rows)

Figure 4. *All eighteen stakeholders with their roles and creation timestamps.*

STAKEHOLDERS JOINED WITH THEIR CERTIFICATION ROWS (LEFT JOIN)

```
SELECT s.id, s.name, s.role, c.certifying_body, c.issue_date, c.expiry_date, c.is_active
FROM stakeholders s
LEFT JOIN certifications c ON c.stakeholder_id = s.id
ORDER BY s.id;
```

id	name	role	certifying_body	issue_date	expiry_date	is_active
1	Green Valley Farm	farmer	FairTrade	2025-01-01	2030-01-01	True
2	Hilltop Coffee Farm	farmer	Organic	2025-01-01	2030-01-01	True
3	Coffee Processors Ltd	processor	Food Safety	2025-01-01	2030-01-01	True
4	Global Coffee Distributors	distributor	Export Cert	2025-01-01	2030-01-01	True
5	City Coffee Retail	retailer	Retail Cert	2025-01-01	2030-01-01	True
6	Cocoa Farms West	farmer	FairTrade	2025-01-01	2030-01-01	True
7	Rainforest Cocoa Ltd	farmer	Organic	2025-01-01	2030-01-01	True
8	Cocoa Processing Hub	processor	Processing Cert	2025-01-01	2030-01-01	True
9	Chocolate Distributors Inc	distributor	Export Cert	2025-01-01	2030-01-01	True
10	Sweet Retail Store	retailer	Retail Cert	2025-01-01	2030-01-01	True
11	Cotton Growers Union	farmer	Organic Cotton	2025-01-01	2030-01-01	True
12	Organic Cotton Farm	farmer	Eco Cert	2025-01-01	2030-01-01	True
13	Textile Processing Ltd	processor	Textile Cert	2025-01-01	2030-01-01	True
14	Fabric Distributors	distributor	Distribution Cert	2025-01-01	2030-01-01	True
15	Clothing Retail Hub	retailer	Retail Cert	2025-01-01	2030-01-01	True
16	Expired Farm	farmer	FairTrade	2018-01-01	2020-01-01	True
17	Uncertified Retailer	retailer	NULL	NULL	NULL	NULL
18	Revoked Distributor	distributor	Export Cert	2025-01-01	2030-01-01	False

(18 rows)

Figure 5. *Stakeholders joined with their certification rows. The LEFT JOIN preserves id 17, whose certification fields are all NULL; the 2020 expiry on id 16 and the* `is_active = False` *on id 18 are clearly visible.*

6.3 Batches and the Supply Chain DAG

Twenty four batches span the three chains. Each batch records its product name, quantity, unit, and the stakeholder currently in custody of it. The `batch_relations` table captures how each batch was produced from its parents, forming the directed acyclic graph that drives all lineage queries.

3 All Product Batches with Owner Details

24 product batches across Coffee (100-107), Cocoa (200-207), and Cotton (300-307) chains.

QUERY

```
SELECT b.id, b.product_name, b.quantity, b.unit, s.name AS owner, s.role
FROM batches b JOIN stakeholders s ON b.current_owner_id = s.id ORDER BY b.id;
```

id	product_name	quantity	unit	owner	role
100	Coffee Raw A	1000	kg	Green Valley Farm	farmer
101	Coffee Raw B	800	kg	Hilltop Coffee Farm	farmer
102	Coffee Proc A1	600	kg	Coffee Processors Ltd	processor
103	Coffee Proc A2	400	kg	Coffee Processors Ltd	processor
104	Coffee Proc B1	800	kg	Coffee Processors Ltd	processor
105	Coffee Export	1800	kg	Global Coffee Distributors	distributor
106	Coffee Retail 1	900	kg	City Coffee Retail	retailer
107	Coffee Retail 2	900	kg	City Coffee Retail	retailer
200	Cocoa Raw A	900	kg	Cocoa Farms West	farmer
201	Cocoa Raw B	700	kg	Rainforest Cocoa Ltd	farmer
202	Cocoa Proc A1	500	kg	Cocoa Processing Hub	processor
203	Cocoa Proc A2	400	kg	Cocoa Processing Hub	processor
204	Cocoa Proc B1	700	kg	Cocoa Processing Hub	processor
205	Cocoa Export	1600	kg	Chocolate Distributors Inc	distributor
206	Chocolate Batch 1	800	kg	Sweet Retail Store	retailer
207	Chocolate Batch 2	800	kg	Sweet Retail Store	retailer
300	Cotton Raw A	1200	kg	Cotton Growers Union	farmer
301	Cotton Raw B	900	kg	Organic Cotton Farm	farmer
302	Cotton Proc A1	700	kg	Textile Processing Ltd	processor
303	Cotton Proc A2	500	kg	Textile Processing Ltd	processor
304	Cotton Proc B1	900	kg	Textile Processing Ltd	processor
305	Fabric Export	2100	kg	Fabric Distributors	distributor
306	Clothing Batch 1	1000	kg	Clothing Retail Hub	retailer
307	Clothing Batch 2	1100	kg	Clothing Retail Hub	retailer

(24 rows)

Figure 6. *All twenty four product batches across the coffee (100 to 107), cocoa (200 to 207), and cotton (300 to 307) chains.*

4 Supply Chain Relationships (Batch Relations)

Parent-child relationships forming a Directed Acyclic Graph (DAG) — models splitting and merging of batches through processing stages.

QUERY

```
SELECT br.parent_batch_id, pb.product_name AS parent_product,
       br.child_batch_id, cb.product_name AS child_product
FROM batch_relations br
JOIN batches pb ON br.parent_batch_id = pb.id
JOIN batches cb ON br.child_batch_id = cb.id
ORDER BY br.parent_batch_id, br.child_batch_id;
```

parent_batch_id	parent_product	child_batch_id	child_product
100	Coffee Raw A	102	Coffee Proc A1
100	Coffee Raw A	103	Coffee Proc A2
101	Coffee Raw B	104	Coffee Proc B1
102	Coffee Proc A1	105	Coffee Export
103	Coffee Proc A2	105	Coffee Export
104	Coffee Proc B1	105	Coffee Export
105	Coffee Export	106	Coffee Retail 1
105	Coffee Export	107	Coffee Retail 2
200	Cocoa Raw A	202	Cocoa Proc A1
200	Cocoa Raw A	203	Cocoa Proc A2
201	Cocoa Raw B	204	Cocoa Proc B1
202	Cocoa Proc A1	205	Cocoa Export
203	Cocoa Proc A2	205	Cocoa Export
204	Cocoa Proc B1	205	Cocoa Export
205	Cocoa Export	206	Chocolate Batch 1
205	Cocoa Export	207	Chocolate Batch 2
300	Cotton Raw A	302	Cotton Proc A1
300	Cotton Raw A	303	Cotton Proc A2
301	Cotton Raw B	304	Cotton Proc B1
302	Cotton Proc A1	305	Fabric Export
303	Cotton Proc A2	305	Fabric Export
304	Cotton Proc B1	305	Fabric Export
305	Fabric Export	306	Clothing Batch 1
305	Fabric Export	307	Clothing Batch 2

(24 rows)

Figure 7. *Parent to child relationships. Each row is one directed edge in the supply chain DAG; merges are visible where two different parents flow into the same child.*

6.4 Recursive Lineage Queries

The core value of the database lies in its lineage queries. Given any batch, the backward trace walks through every ancestor up to the raw material origin, and the forward trace walks through every descendant to the final retail product. Both are implemented with a PostgreSQL recursive CTE that joins `batch_relations` back onto itself.

5 Backward Lineage Tracing — Coffee Retail to Origin

Traces *Coffee Retail 1* (batch 106) backward through every supply-chain stage to its raw-material origins at the farm level. The recursive CTE walks the `batch_relations` DAG until no more parents are found; `UNION` (not `UNION ALL`) collapses the same node visited through multiple paths.

RECURSIVE BACKWARD LINEAGE QUERY (BATCH_ID = 106)

```
WITH RECURSIVE lineage AS (
    SELECT b.id, b.product_name, s.name AS owner, s.role, 0 AS depth
    FROM batches b JOIN stakeholders s ON b.current_owner_id = s.id
    WHERE b.id = 106
    UNION
    SELECT parent.id, parent.product_name, ps.name, ps.role, l.depth + 1
    FROM batches parent
    JOIN stakeholders ps ON parent.current_owner_id = ps.id
    JOIN batch_relations br ON br.parent_batch_id = parent.id
    JOIN lineage l ON l.id = br.child_batch_id
)
SELECT * FROM lineage ORDER BY depth DESC, id;
```

id	product_name	owner	role	depth
100	Coffee Raw A	Green Valley Farm	farmer	3
101	Coffee Raw B	Hilltop Coffee Farm	farmer	3
102	Coffee Proc A1	Coffee Processors Ltd	processor	2
103	Coffee Proc A2	Coffee Processors Ltd	processor	2
104	Coffee Proc B1	Coffee Processors Ltd	processor	2
105	Coffee Export	Global Coffee Distributors	distributor	1
106	Coffee Retail 1	City Coffee Retail	retailer	0

(7 rows)

Figure 8. *Backward lineage starting at Coffee Retail 1 (batch 106). The recursive CTE walks back through distributor, processor, and the two farm level raw batches. The depth column shows distance from the target batch.*

6 Forward Tracing — Impact Analysis from Raw Material

Simulates a recall: traces *Cocoa Raw A* (batch 200) forward to find every downstream product that would be affected. Critical for rapid impact analysis.

RECURSIVE FORWARD TRACE QUERY (BATCH_ID = 200)

```
WITH RECURSIVE trace AS (
    SELECT b.id, b.product_name, s.name AS owner, s.role, 0 AS depth
    FROM batches b JOIN stakeholders s ON b.current_owner_id = s.id
    WHERE b.id = 200
    UNION
    SELECT child.id, child.product_name, cs.name, cs.role, t.depth + 1
    FROM batches child
    JOIN stakeholders cs ON child.current_owner_id = cs.id
    JOIN batch_relations br ON br.child_batch_id = child.id
    JOIN trace t ON t.id = br.parent_batch_id
)
SELECT * FROM trace ORDER BY depth, id;
```

id	product_name	owner	role	depth
200	Cocoa Raw A	Cocoa Farms West	farmer	0
202	Cocoa Proc A1	Cocoa Processing Hub	processor	1
203	Cocoa Proc A2	Cocoa Processing Hub	processor	1
205	Cocoa Export	Chocolate Distributors Inc	distributor	2
206	Chocolate Batch 1	Sweet Retail Store	retailer	3
207	Chocolate Batch 2	Sweet Retail Store	retailer	3

(6 rows)

Figure 9. *Forward trace starting at Cocoa Raw A (batch 200). Every downstream product that carries material from this raw batch is returned. This is the query a quality manager would run during a product recall.*

6.5 Trigger Enforcement

Four separate scenarios verify that the two certification triggers reject every non-compliant action. In each case the database returns an error and the row is never written. The application client cannot bypass these checks because they execute inside the database engine.

7 Certification Trigger — Blocking Uncertified Stakeholder

The `certification_trigger` (BEFORE INSERT on `batches`) prevents any batch creation if the stakeholder lacks a valid active certification. Stakeholder 17 (*Uncertified Retailer*) has **no row** in the certifications table.

ATTEMPT: INSERT BATCH FOR UNCERTIFIED RETAILER (STAKEHOLDER_ID = 17)

```
INSERT INTO batches (product_name, quantity, unit, current_owner_id) VALUES ('Illegal Batch', 100, 'kg', 17);
```

```
X ERROR: Stakeholder is not certified or certification expired CONTEXT: PL/pgSQL function check_certification() line 9 at RAISE
```

8 Certification Trigger — Blocking Expired Certification

Stakeholder 16 (*Expired Farm*) holds a certification whose `expiry_date = 2020-01-01`. Because `expiry_date < CURRENT_DATE`, the trigger rejects the INSERT.

ATTEMPT: INSERT BATCH FOR EXPIRED FARM (STAKEHOLDER_ID = 16)

```
INSERT INTO batches (product_name, quantity, unit, current_owner_id) VALUES ('Expired Cert Batch', 100, 'kg', 16);
```

```
X ERROR: Stakeholder is not certified or certification expired CONTEXT: PL/pgSQL function check_certification() line 9 at RAISE
```

VERIFYING THE STAKEHOLDER'S CERTIFICATION (EXPIRY IN THE PAST)

```
SELECT id, stakeholder_id, certifying_body, issue_date, expiry_date, is_active FROM certifications WHERE stakeholder_id = 16;
```

id	stakeholder_id	certifying_body	issue_date	expiry_date	is_active
16	16	FairTrade	2018-01-01	2020-01-01	True

(1 row)

Figure 10. *Top: attempt to insert a batch for stakeholder 17 (no certification on file). Bottom: attempt for stakeholder 16, whose certification expired on 2020-01-01. Both inserts are rejected by* `check_certification()` *.*

9 Certification Trigger — Blocking Revoked Certification

Stakeholder 18 (*Revoked Distributor*) holds a certification that was administratively revoked (`is_active = FALSE`). The trigger rejects the INSERT even though the expiry date is in the future.

ATTEMPT: INSERT BATCH FOR REVOKED DISTRIBUTOR (STAKEHOLDER_ID = 18)

```
INSERT INTO batches (product_name, quantity, unit, current_owner_id) VALUES ('Revoked Cert Batch', 100, 'kg', 18);
```

```
✗ ERROR: Stakeholder is not certified or certification expired CONTEXT: PL/pgSQL function check_certification() line 9 at RAISE
```

VERIFYING THE STAKEHOLDER'S CERTIFICATION (IS_ACTIVE = FALSE)

```
SELECT id, stakeholder_id, certifying_body, issue_date, expiry_date, is_active FROM certifications WHERE stakeholder_id = 18;
```

id	stakeholder_id	certifying_body	issue_date	expiry_date	is_active
17	18	Export Cert	2025-01-01	2030-01-01	False

(1 row)

10 Transfer Trigger — Blocking Transfer to Uncertified Owner

The `transfer_certification_trigger` (BEFORE UPDATE on `batches`) fires on ownership change only. It guarantees that a certified batch cannot be moved into the custody of an uncertified stakeholder, closing the gap that the INSERT trigger alone would miss.

ATTEMPT: TRANSFER BATCH 107 TO UNCERTIFIED RETAILER (STAKEHOLDER_ID = 17)

```
UPDATE batches SET current_owner_id = 17 WHERE id = 107;
```

```
✗ ERROR: Transfer denied: receiving stakeholder (id=17) is not certified or certification has expired. CONTEXT: PL/pgSQL function check_transfer_certification() line 10 at RAISE
```

Figure 11. *Top: attempt to insert a batch for stakeholder 18, whose certification has* `is_active = FALSE`*. Bottom: attempt to transfer an existing batch to uncertified stakeholder 17; the second trigger* `check_transfer_certification()` *fires on UPDATE of* `current_owner_id` *and blocks the move.*

6.6 Transfer and Audit Logging

A successful transfer is performed through the `transfer_batch(batch_id, new_owner_id)` stored function. The function updates the ownership column, the transfer trigger validates the receiving stakeholder, and the audit trigger records the event automatically. The audit log then becomes the canonical record of who held the batch and when.

11 Batch Transfer with Automatic Audit Logging

Demonstrates the `transfer_batch(batch_id, new_owner_id)` stored function, which safely re-assigns ownership. The `audit_update_trigger` automatically logs a `transferred` row, and the `transfer_certification_trigger` validates that the receiving stakeholder is certified.

BEFORE TRANSFER — BATCH 106

```
SELECT b.id, b.product_name, s.name AS current_owner FROM batches b JOIN stakeholders s ON b.current_owner_id = s.id WHERE b.id = 106;
```

id	product_name	current_owner
106	Coffee Retail 1	City Coffee Retail

(1 row)

EXECUTE TRANSFER FUNCTION

```
SELECT transfer_batch(106, 4);
```

transfer_batch
Batch "Coffee Retail 1" (id=106) transferred from City Coffee Retail to Global Coffee Distributors.

(1 row)

AFTER TRANSFER — BATCH 106

```
SELECT b.id, b.product_name, s.name AS current_owner FROM batches b JOIN stakeholders s ON b.current_owner_id = s.id WHERE b.id = 106;
```

id	product_name	current_owner
106	Coffee Retail 1	Global Coffee Distributors

(1 row)

AUDIT TRAIL FOR BATCH 106 (SHOWING TRANSFER LOGGED AUTOMATICALLY)

```
SELECT * FROM v_audit_trail WHERE batch_id = 106 ORDER BY timestamp DESC LIMIT 3;
```

log_id	action	batch_id	product_name	actor_name	actor_role	timestamp
25	transferred	106	Coffee Retail 1	Global Coffee Distributors	distributor	2026-04-19 14:59:22.407963
7	created	106	Coffee Retail 1	City Coffee Retail	retailer	2026-04-19 14:54:47.986876

(2 rows)

Figure 12. *Batch 106 is transferred from City Coffee Retail back to Global Coffee Distributors. The before and after states are shown alongside the audit trail, which now contains a new row of action* `transferred` *written by the trigger with no application assistance.*

12 Audit Log Immutability — Tamper-Proof Verification

The `audit_immutability_trigger` rejects any UPDATE or DELETE on `audit_log`, guaranteeing a tamper-proof, append-only transaction ledger for third-party auditing.

ATTEMPT: MODIFY AN AUDIT RECORD

```
UPDATE audit_log SET action = 'hacked' WHERE id = 1;
```

```
X ERROR: Audit log records are immutable and cannot be modified or deleted. CONTEXT: PL/pgSQL function prevent_audit_modification() line 3 at RAISE
```

ATTEMPT: DELETE AN AUDIT RECORD

```
DELETE FROM audit_log WHERE id = 1;
```

```
X ERROR: Audit log records are immutable and cannot be modified or deleted. CONTEXT: PL/pgSQL function prevent_audit_modification() line 3 at RAISE
```

Figure 13. *Audit log immutability in action. Both UPDATE and DELETE against `audit_log` are rejected by `prevent_audit_modification()`, guaranteeing an append only history.*

6.7 Views

The database exposes six views that pre-compose common queries. Three of the most important are shown below: the full compliance overview, the per chain summary, and the final products and raw materials views used for recall and provenance queries respectively.

13 Views — Compliance Overview (all four states)

The `v_compliance_overview` view provides a real-time compliance status for every stakeholder. Thanks to the edge-case seed data, all four CASE branches appear: `COMPLIANT`, `EXPIRED` (id 16), `NO CERTIFICATION` (id 17), and `REVOKED` (id 18).

QUERY

```
SELECT * FROM v_compliance_overview ORDER BY stakeholder_id;
```

stakeholder_id	name	role	certifying_body	issue_date	expiry_date	is_active	compliance_status
1	Green Valley Farm	farmer	FairTrade	2025-01-01	2030-01-01	True	COMPLIANT
2	Hilltop Coffee Farm	farmer	Organic	2025-01-01	2030-01-01	True	COMPLIANT
3	Coffee Processors Ltd	processor	Food Safety	2025-01-01	2030-01-01	True	COMPLIANT
4	Global Coffee Distributors	distributor	Export Cert	2025-01-01	2030-01-01	True	COMPLIANT
5	City Coffee Retail	retailer	Retail Cert	2025-01-01	2030-01-01	True	COMPLIANT
6	Cocoa Farms West	farmer	FairTrade	2025-01-01	2030-01-01	True	COMPLIANT
7	Rainforest Cocoa Ltd	farmer	Organic	2025-01-01	2030-01-01	True	COMPLIANT
8	Cocoa Processing Hub	processor	Processing Cert	2025-01-01	2030-01-01	True	COMPLIANT
9	Chocolate Distributors Inc	distributor	Export Cert	2025-01-01	2030-01-01	True	COMPLIANT
10	Sweet Retail Store	retailer	Retail Cert	2025-01-01	2030-01-01	True	COMPLIANT
11	Cotton Growers Union	farmer	Organic Cotton	2025-01-01	2030-01-01	True	COMPLIANT
12	Organic Cotton Farm	farmer	Eco Cert	2025-01-01	2030-01-01	True	COMPLIANT
13	Textile Processing Ltd	processor	Textile Cert	2025-01-01	2030-01-01	True	COMPLIANT
14	Fabric Distributors	distributor	Distribution Cert	2025-01-01	2030-01-01	True	COMPLIANT
15	Clothing Retail Hub	retailer	Retail Cert	2025-01-01	2030-01-01	True	COMPLIANT
16	Expired Farm	farmer	FairTrade	2018-01-01	2020-01-01	True	EXPIRED
17	Uncertified Retailer	retailer	NULL	NULL	NULL	NULL	NO CERTIFICATION
18	Revoked Distributor	distributor	Export Cert	2025-01-01	2030-01-01	False	REVOKED

(18 rows)

Figure 14. *`v_compliance_overview` classifies every stakeholder into one of four states. Thanks to the edge case seed data, all four CASE branches are reached: COMPLIANT (rows 1 to 15), EXPIRED (row 16), NO CERTIFICATION (row 17), and REVOKED (row 18).*

14 Views — Supply Chain Summary

Aggregated statistics per product supply chain — total batches, total quantity, and unique stakeholders involved.

QUERY

```
SELECT * FROM v_supply_chain_summary;
```

supply_chain	total_batches	total_quantity	unique_stakeholders
Cocoa	8	6400	5
Coffee	8	7200	5
Cotton	8	8400	5

(3 rows)

Figure 15. `v_supply_chain_summary` *aggregates per chain: eight batches, five stakeholders, and a total quantity that reflects the in-transit material in each of the three chains.*

15 Views — Final Retail Products & Raw Materials

v_final_products shows leaf batches (retail-ready, no downstream children). v_raw_materials shows root batches (farm-origin, no upstream parents).

FINAL RETAIL PRODUCTS (LEAF NODES)

```
SELECT * FROM v_final_products ORDER BY batch_id;
```

batch_id	product_name	amount	retailer
106	Coffee Retail 1	900 kg	City Coffee Retail
107	Coffee Retail 2	900 kg	City Coffee Retail
206	Chocolate Batch 1	800 kg	Sweet Retail Store
207	Chocolate Batch 2	800 kg	Sweet Retail Store
306	Clothing Batch 1	1000 kg	Clothing Retail Hub
307	Clothing Batch 2	1100 kg	Clothing Retail Hub

(6 rows)

RAW MATERIAL SOURCES (ROOT NODES)

```
SELECT * FROM v_raw_materials ORDER BY batch_id;
```

batch_id	product_name	amount	source_farm
100	Coffee Raw A	1000 kg	Green Valley Farm
101	Coffee Raw B	800 kg	Hilltop Coffee Farm
200	Cocoa Raw A	900 kg	Cocoa Farms West
201	Cocoa Raw B	700 kg	Rainforest Cocoa Ltd
300	Cotton Raw A	1200 kg	Cotton Growers Union
301	Cotton Raw B	900 kg	Organic Cotton Farm

(6 rows)

Figure 16. `v_final_products` *(top) lists the leaf batches that have no downstream children, and* `v_raw_materials` *(bottom) lists the root batches that have no upstream parents. Together they bound the DAG.*

6.8 Stored Functions

The three reusable lineage and compliance functions let client applications call a single routine instead of rewriting the recursive CTEs each time.

16 Stored Function — get_backward_lineage()

The get_backward_lineage() stored function wraps the recursive CTE into a reusable call. Tracing Chocolate Batch 2 (id=207) back to cocoa farm origins.

FUNCTION CALL

```
SELECT * FROM get_backward_lineage(207);
```

batch_id	product_name	owner_name	owner_role	depth
200	Cocoa Raw A	Cocoa Farms West	farmer	3
201	Cocoa Raw B	Rainforest Cocoa Ltd	farmer	3
204	Cocoa Proc B1	Cocoa Processing Hub	processor	2
203	Cocoa Proc A2	Cocoa Processing Hub	processor	2
202	Cocoa Proc A1	Cocoa Processing Hub	processor	2
205	Cocoa Export	Chocolate Distributors Inc	distributor	1
207	Chocolate Batch 2	Sweet Retail Store	retailer	0

(7 rows)

Figure 17. `get_backward_lineage(207)` *wraps the recursive CTE used in Figure 8 into a callable function. Here it traces Chocolate Batch 2 back to both cocoa farm origins.*

17 Stored Function — get_forward_trace()

The get_forward_trace() function traces Cotton Raw A (id=300) forward to find all downstream clothing products — useful for recall/impact analysis.

FUNCTION CALL

```
SELECT * FROM get_forward_trace(300);
```

batch_id	product_name	owner_name	owner_role	depth
300	Cotton Raw A	Cotton Growers Union	farmer	0
302	Cotton Proc A1	Textile Processing Ltd	processor	1
303	Cotton Proc A2	Textile Processing Ltd	processor	1
305	Fabric Export	Fabric Distributors	distributor	2
306	Clothing Batch 1	Clothing Retail Hub	retailer	3
307	Clothing Batch 2	Clothing Retail Hub	retailer	3

(6 rows)

18 Stored Function — get_compliance_report()

Full compliance audit. Its result shape matches `v_compliance_overview`, so the function and the view are interchangeable. Stakeholders 16, 17, and 18 are flagged as non-compliant (EXPIRED, NO CERTIFICATION, REVOKED respectively).

FUNCTION CALL

```
SELECT * FROM get_compliance_report();
```

stakeholder_id	name	role	certifying_body	issue_date	expiry_date	is_active	compliance_status
1	Green Valley Farm	farmer	FairTrade	2025-01-01	2030-01-01	True	COMPLIANT
2	Hilltop Coffee Farm	farmer	Organic	2025-01-01	2030-01-01	True	COMPLIANT
3	Coffee Processors Ltd	processor	Food Safety	2025-01-01	2030-01-01	True	COMPLIANT
4	Global Coffee Distributors	distributor	Export Cert	2025-01-01	2030-01-01	True	COMPLIANT
5	City Coffee Retail	retailer	Retail Cert	2025-01-01	2030-01-01	True	COMPLIANT
6	Cocoa Farms West	farmer	FairTrade	2025-01-01	2030-01-01	True	COMPLIANT
7	Rainforest Cocoa Ltd	farmer	Organic	2025-01-01	2030-01-01	True	COMPLIANT
8	Cocoa Processing Hub	processor	Processing Cert	2025-01-01	2030-01-01	True	COMPLIANT
9	Chocolate Distributors Inc	distributor	Export Cert	2025-01-01	2030-01-01	True	COMPLIANT
10	Sweet Retail Store	retailer	Retail Cert	2025-01-01	2030-01-01	True	COMPLIANT
11	Cotton Growers Union	farmer	Organic Cotton	2025-01-01	2030-01-01	True	COMPLIANT
12	Organic Cotton Farm	farmer	Eco Cert	2025-01-01	2030-01-01	True	COMPLIANT
13	Textile Processing Ltd	processor	Textile Cert	2025-01-01	2030-01-01	True	COMPLIANT
14	Fabric Distributors	distributor	Distribution Cert	2025-01-01	2030-01-01	True	COMPLIANT
15	Clothing Retail Hub	retailer	Retail Cert	2025-01-01	2030-01-01	True	COMPLIANT
16	Expired Farm	farmer	FairTrade	2018-01-01	2020-01-01	True	EXPIRED
17	Uncertified Retailer	retailer	NULL	NULL	NULL	NULL	NO CERTIFICATION
18	Revoked Distributor	distributor	Export Cert	2025-01-01	2030-01-01	False	REVOKED

(18 rows)

Figure 18. *Top: `get_forward_trace(300)` traces Cotton Raw A forward to every downstream fabric and clothing product. Bottom: `get_compliance_report()` returns the same result set as `v_compliance_overview`, confirming that the view and the function are interchangeable.*

6.9 Analytical Queries

Aggregate queries exercise GROUP BY, HAVING, and a second independent recursive CTE that computes the maximum depth of each chain from its root. These demonstrate that the schema supports realistic analytical workloads in addition to the lineage queries.

19 Analytical Queries — Aggregates & Statistics

Aggregate and analytical queries demonstrating GROUP BY, HAVING, recursive depth calculation, and audit statistics.

TOTAL QUANTITY PER SUPPLY CHAIN STAGE

```
SELECT s.role, SUM(b.quantity) AS total_quantity, COUNT(b.id) AS batch_count
FROM batches b JOIN stakeholders s ON b.current_owner_id = s.id
GROUP BY s.role
ORDER BY CASE s.role WHEN 'farmer' THEN 1 WHEN 'processor' THEN 2 WHEN 'distributor' THEN 3 WHEN 'retailer' THEN 4 END;
```

role	total_quantity	batch_count
farmer	5500	6
processor	5500	9
distributor	5500	3
retailer	5500	6

(4 rows)

BRANCHING FACTOR — BATCHES WITH MOST CHILDREN

```
SELECT b.id, b.product_name, COUNT(br.child_batch_id) AS children_count
FROM batches b LEFT JOIN batch_relations br ON b.id = br.parent_batch_id
GROUP BY b.id, b.product_name HAVING COUNT(br.child_batch_id) > 0
ORDER BY children_count DESC;
```

id	product_name	children_count
305	Fabric Export	3
205	Cocoa Export	2
300	Cotton Raw A	2
105	Coffee Export	2
200	Cocoa Raw A	2
100	Coffee Raw A	2
102	Coffee Proc A1	1
103	Coffee Proc A2	1
204	Cocoa Proc B1	1
104	Coffee Proc B1	1
303	Cotton Proc A2	1
202	Cocoa Proc A1	1
301	Cotton Raw B	1
201	Cocoa Raw B	1
304	Cotton Proc B1	1
101	Coffee Raw B	1
302	Cotton Proc A1	1
203	Cocoa Proc A2	1

(15 rows)

Figure 19. *Total quantity per supply chain stage and the branching factor per batch. The branching factor view highlights which batches split into the largest number of children, which is useful for risk analysis.*

SUPPLY CHAIN DEPTH PER ROOT BATCH

```
WITH RECURSIVE chain AS (
    SELECT b.id AS root_id, b.product_name AS root_product, b.id, 0 AS depth
    FROM batches b WHERE b.id NOT IN (SELECT child_batch_id FROM batch_relations)
    UNION ALL
    SELECT c.root_id, c.root_product, br.child_batch_id, c.depth + 1
    FROM chain c JOIN batch_relations br ON br.parent_batch_id = c.id
)
SELECT root_id, root_product, MAX(depth) AS max_chain_depth
FROM chain GROUP BY root_id, root_product ORDER BY root_id;
```

root_id	root_product	max_chain_depth
100	Coffee Raw A	3
101	Coffee Raw B	3
200	Cocoa Raw A	3
201	Cocoa Raw B	3
300	Cotton Raw A	3
301	Cotton Raw B	3

(6 rows)

AUDIT ACTIONS SUMMARY

```
SELECT action, COUNT(*) AS total FROM audit_log GROUP BY action ORDER BY total DESC;
```

action	total
created	24

(1 row)

Figure 20. *Supply chain depth per root batch and a summary of audit actions. Every chain has a maximum depth of three, consistent with the farmer, processor, distributor, and retailer stages.*

6.10 Supply Chain Visualisation

The final visualisation is generated by `visualise/visualise_db.py`, which reads the current state of the database and emits a Graphviz rendering of the entire DAG. Node colours follow stakeholder role, making the four supply chain stages visually distinct.

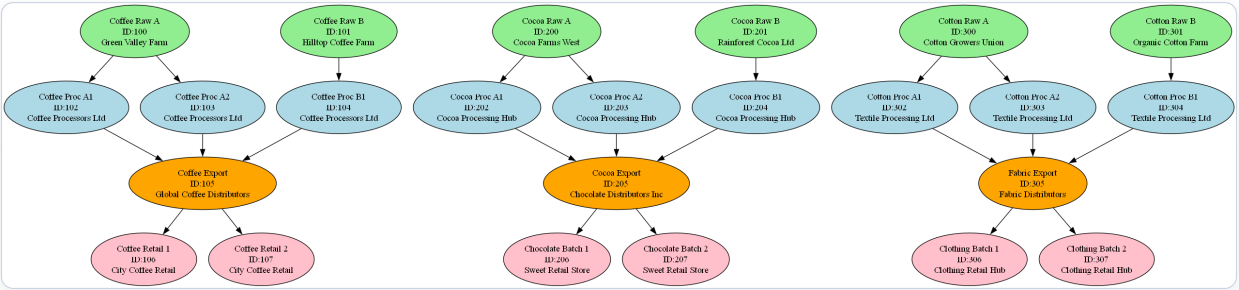


Figure 21. *The complete supply chain graph produced from live data. Green nodes are farmer owned raw batches, blue are processor batches, orange are distributor export batches, and pink are retail ready final products. The three visually separate subgraphs are the coffee, cocoa, and cotton chains, each branching and merging through the processing stages.*

6.11 Summary of Results

All functional objectives defined in Section 2 have been met. The triggers block every non-compliant insert and every non-compliant transfer; the audit log is strictly append only; the recursive functions correctly return the ancestors and descendants for every tested batch; the compliance view returns all four categorical states; and the Python reporting layer produces a complete run report together with the live ER diagram and the supply chain DAG. The schema is in BCNF, which removes update, insert, and delete anomalies from the design.

Rushil Jain (241EC148) · Shresh Parti (241EC252)